

2.3 User Interfaces

Menus, Notifications and Design for Everyone

Objectives

Create Menus (ToolBar)

Use Toasts, Snackbar and Notifications

Design UI for everyone

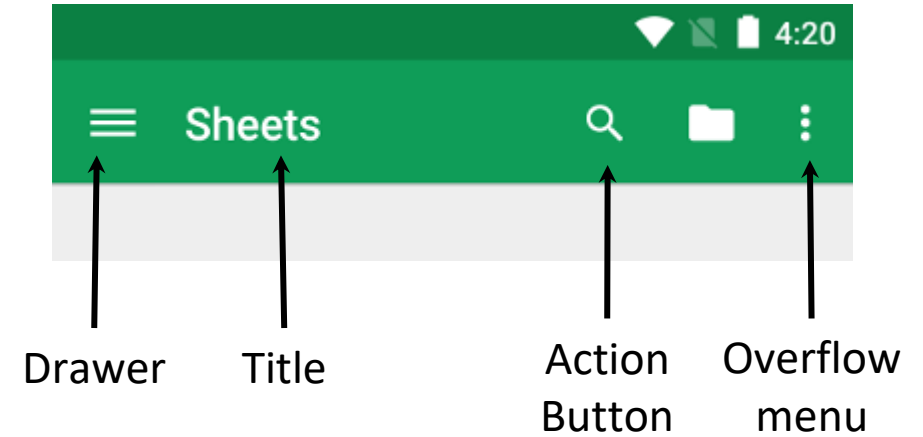
Make apps more accessible

Menus

Menus

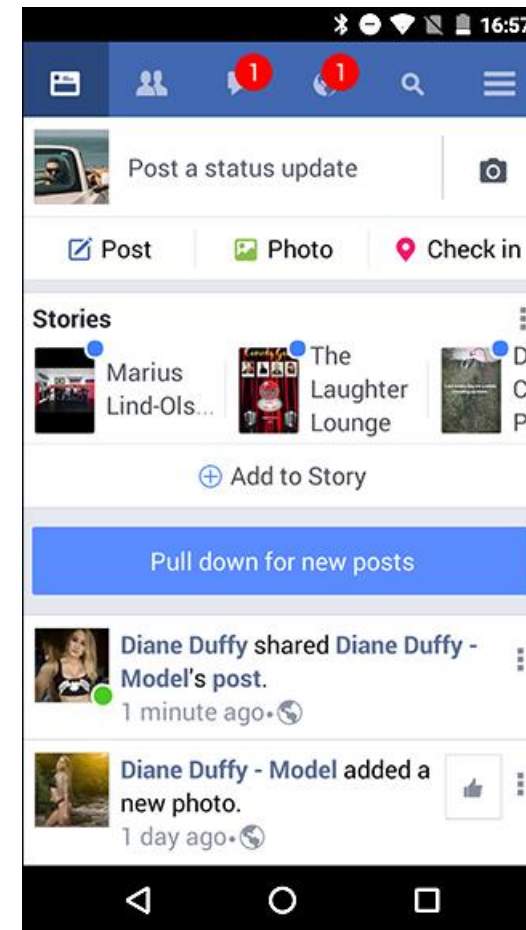
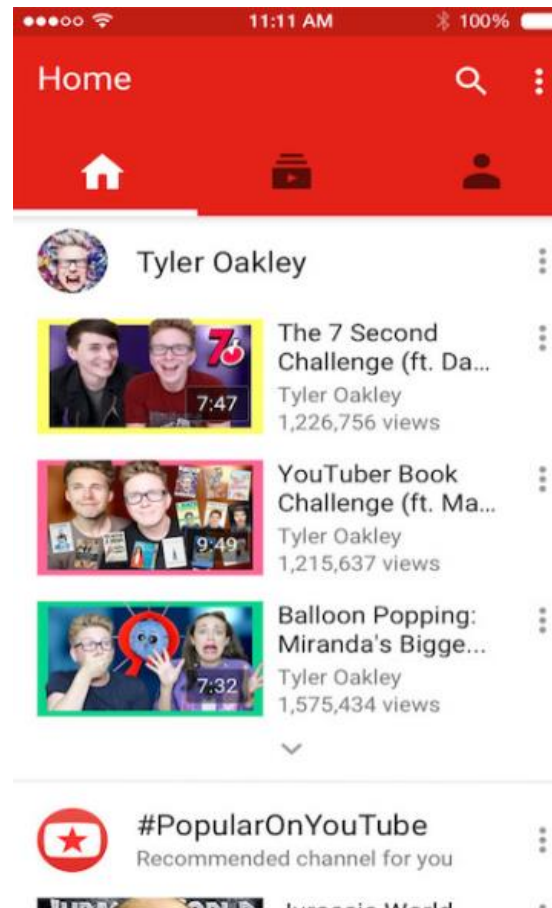
Presents actions and options to user

Presented using the Toolbar widget



Question?

Why use the Menus?

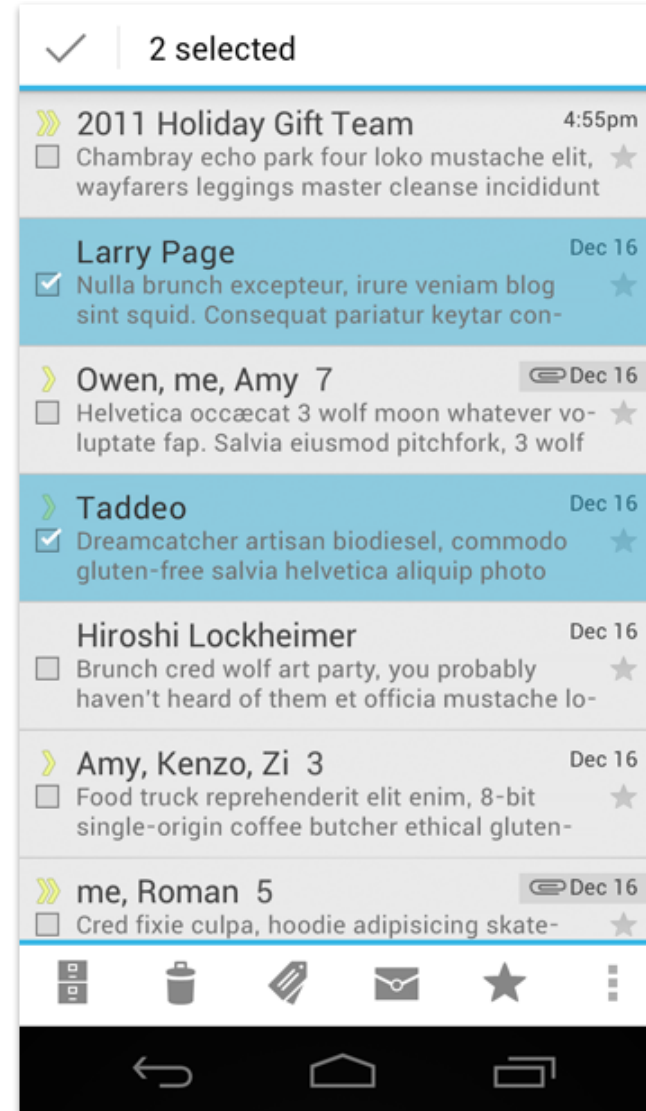
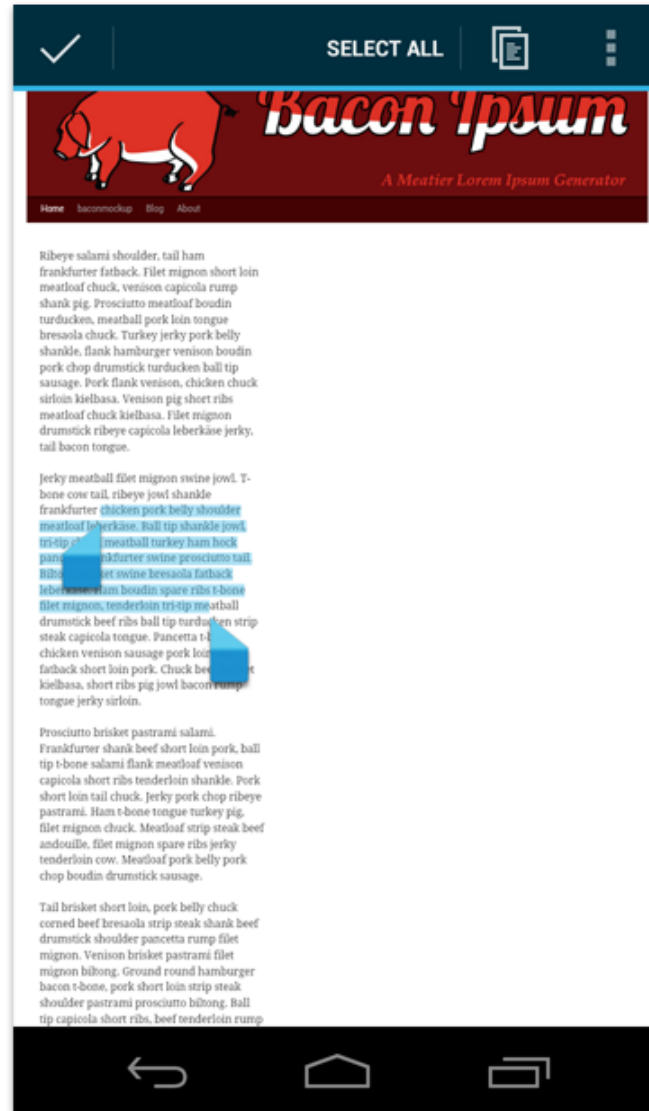


Menus

For guidance on prioritizing actions, use the FIT scheme:

- a. Frequent
- b. Important
- c. Typical

Context Sensitive Menus



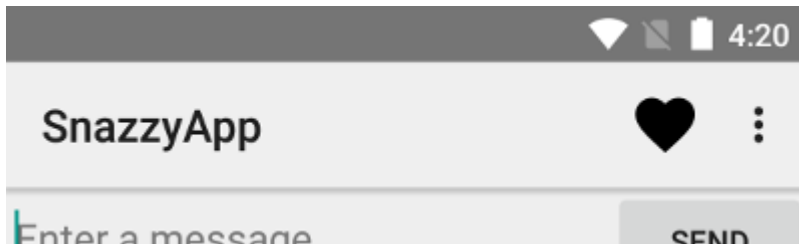
Specify the Actions in XML

```
<menu xmlns:android="http://schemas.android.com/apk/res/android" >

    <!-- "Mark Favorite", should appear as action button if possible -->
    <item
        android:id="@+id/action_favorite"
        android:icon="@drawable/ic_favorite_black_48dp"
        android:title="@string/action_favorite"
        app:showAsAction="ifRoom"/>

    <!-- Settings, should always be in the overflow -->
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
        app:showAsAction="never"/>

</menu>
```



Menus in Jetpack Compose

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyAppBar() {
    TopAppBar(
        title = { Text("My Toolbar") },
        navigationIcon = {
            IconButton(onClick = { /* Handle navigation click */ }) {
                Icon(Icons.Default.Menu, contentDescription = "Menu")
            }
        },
        actions = {
            IconButton(onClick = { /* Handle search */ }) {
                Icon(Icons.Default.Search, contentDescription = "Search")
            }
            IconButton(onClick = { /* Handle settings */ }) {
                Icon(Icons.Default.Settings, contentDescription = "Settings")
            }
        }
    )
}
```

AppBar Placement

- AppBar should be placed inside a Scaffold

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            MyApplicationTheme {  
                Scaffold(topBar = {MyAppBar()},  
                    modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```

Download action bar icons

To best match the Android iconography guidelines, you should use icons provided in the Material icons site

<https://material.io/icons/>

Jetpack compose

```
Icon(Icons.Default.Search, contentDescription = "Search")
```



Adding an Up Button

Helps users to find their way back to the app's main screen

A child activity must declare its parent in the manifest file

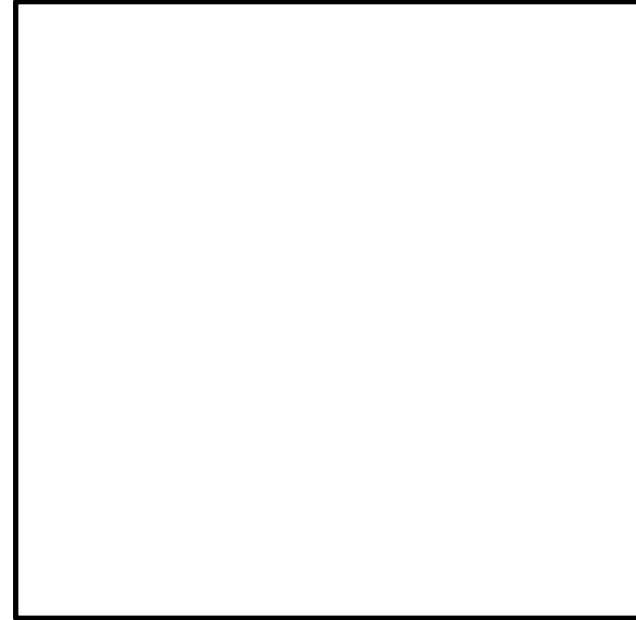


Add Up Button for Low-level Activities

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyTopAppBar(navController: NavController) {
    TopAppBar(
        title = { Text("Details Screen") },
        navigationIcon = {
            IconButton(onClick = { navController.navigateUp() }) { // Handle back navigation
                Icon(Icons.Default.ArrowBack, contentDescription = "Back")
            }
        }
    )
}
```

Question?

1. Why the Menu (Toolbar) is an important UI component for a mobile app?
2. What is the main purpose of setting up the Up button?



Toasts, Snack bar and Notifications

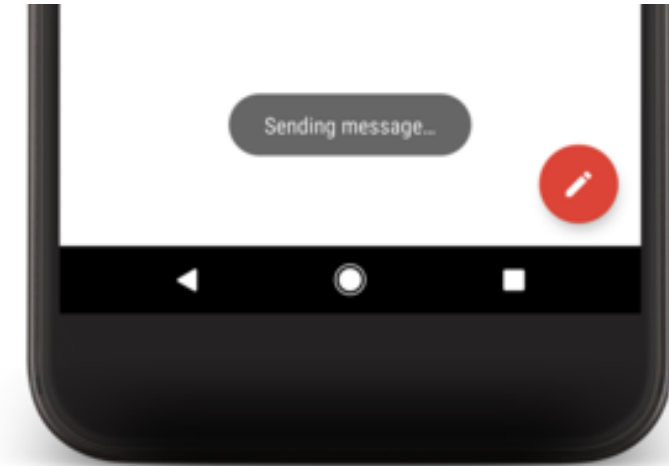
Toast

Simple feedback about an operation in a small popup

It only fills the amount of space required for the message

```
val text = "Hello toast!"
```

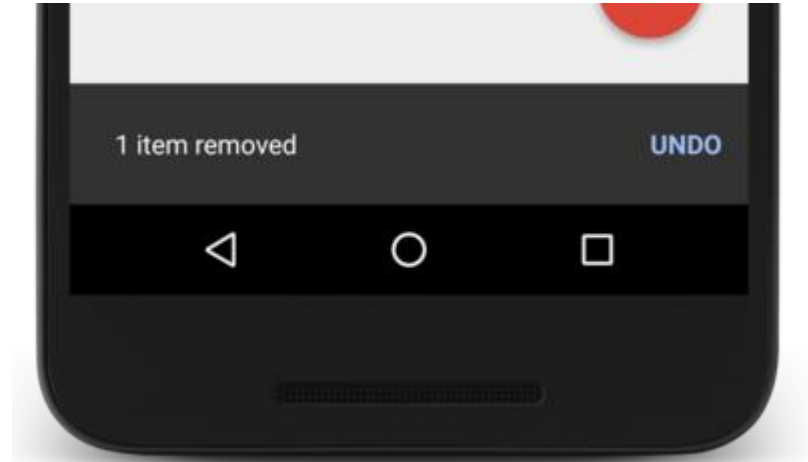
```
Toast.makeText(context, text, Toast.LENGTH_SHORT).show()
```



Snackbar

Displays a brief message

Action could be added to snackbar



```
val mySnackbar = Snackbar.make(findViewById(R.id.myCoordinatorLayout),  
                                R.string.email_archived,  
                                Snackbar.LENGTH_SHORT)
```

```
mySnackbar.setAction(R.string.undo_string, MyUndoListener())
```

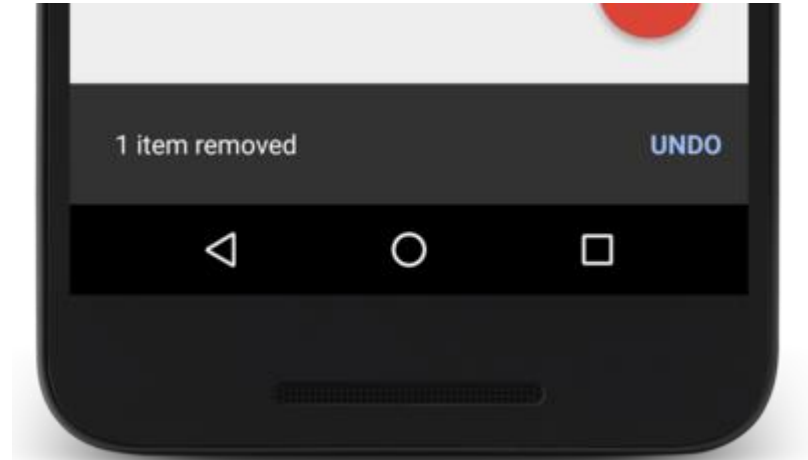
```
mySnackbar.show()
```

<https://developer.android.com/training/snackbar/action>

Jetpack Compose

```
@Composable
fun SnackbarExample() {
    val snackbarHostState = remember { SnackbarHostState() }
    val coroutineScope = rememberCoroutineScope()

    Scaffold(
        snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
    ) { padding ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            contentAlignment = Alignment.Center
        ) {
            Button(onClick = {
                coroutineScope.launch {
                    snackbarHostState.showSnackbar(
                        message = "This is a Snackbar",
                        actionLabel = "Dismiss",
                        duration = SnackbarDuration.Short
                    )
                }
            }) {
                Text("Show Snackbar")
            }
        }
    }
}
```



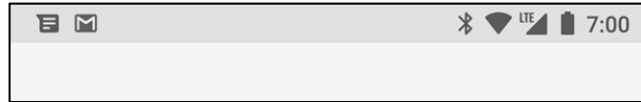
Notification

A message displays outside your app's UI

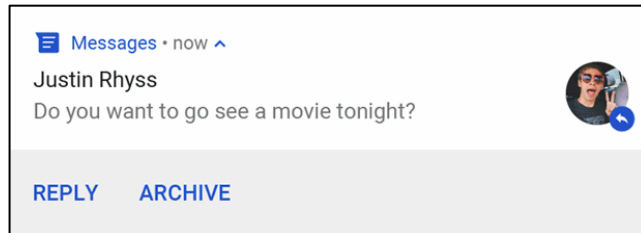
Purposes:

- Reminder
- Communication from other people
- Other timely information from your app

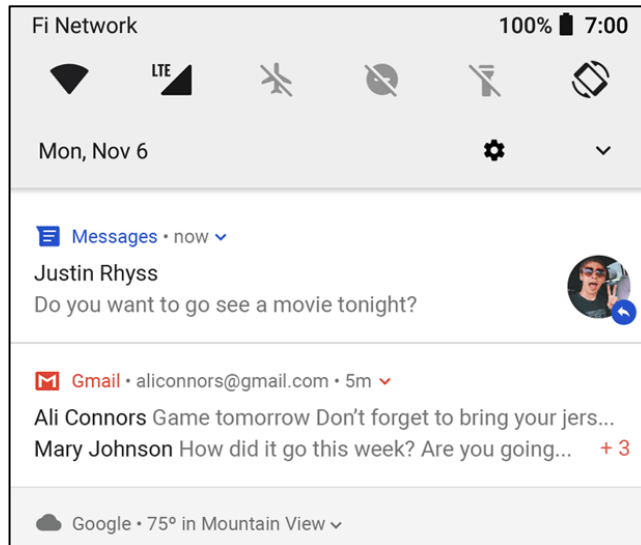
Notification



Notification icons appear on the left side of the status bar



A heads-up notification appears in front of the foreground app

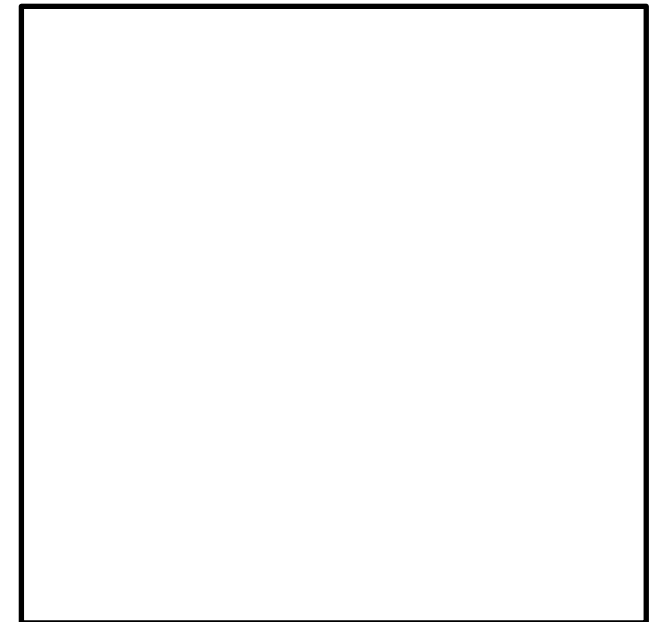


Notifications in the notification drawer

Find out ways to create a notification here : <https://developer.android.com/training/notify-user/build-notification>

Question?

1. When a Toast is more appropriate compared to a Notification?
2. Among Notification and Snackbar, which one should be used to perform the following tasks:
 - a. Inform user that the phone internal storage is low
 - b. A message has been deleted from In box
 - c. An update has been made available



Design for Everyone

Design for Everyone

Android devices come in many shapes and sizes all around the world

Device configurations:

- Languages
- Screen
- Versions of Android

Design for Everyone

Languages

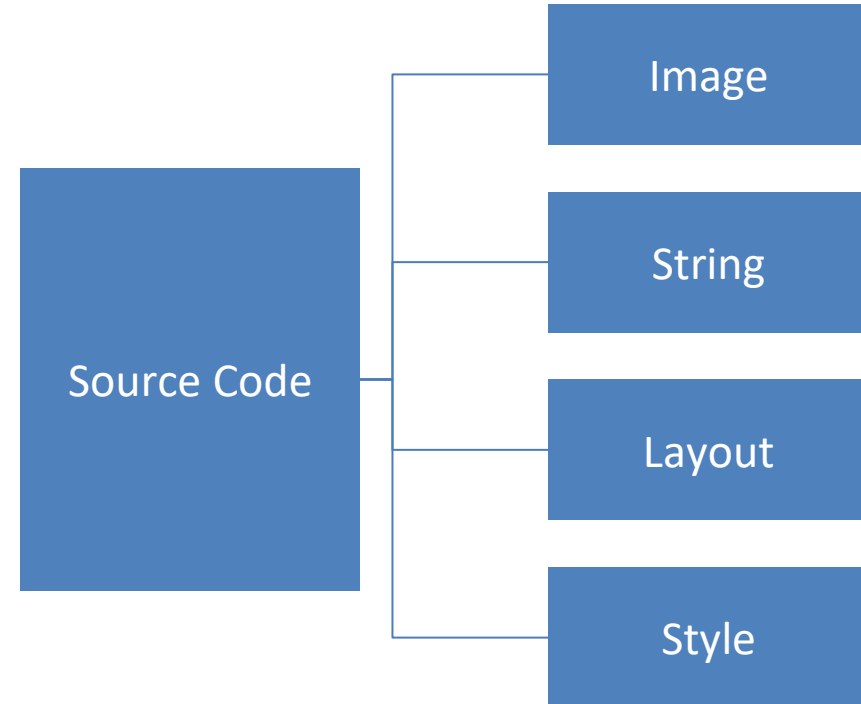
Screen

Versions of Android

Techniques

Externalize resources

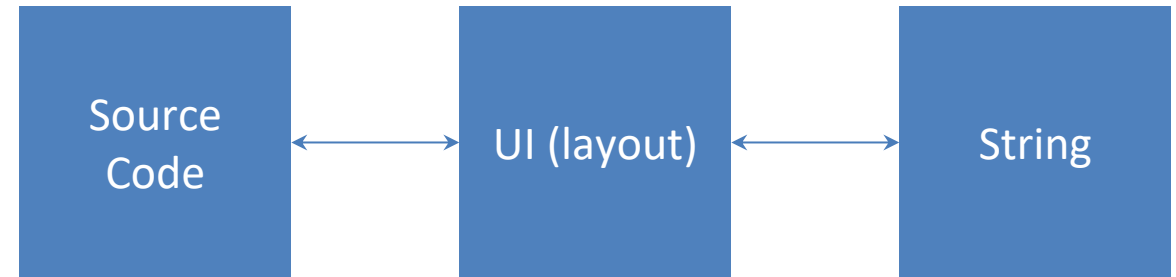
Maintain resources
independently



Supporting Different Languages

Extract UI strings from your app code and keep them in an external file.

The `res/values/strings.xml` holds your string values.



Create Locale Directories and String Files

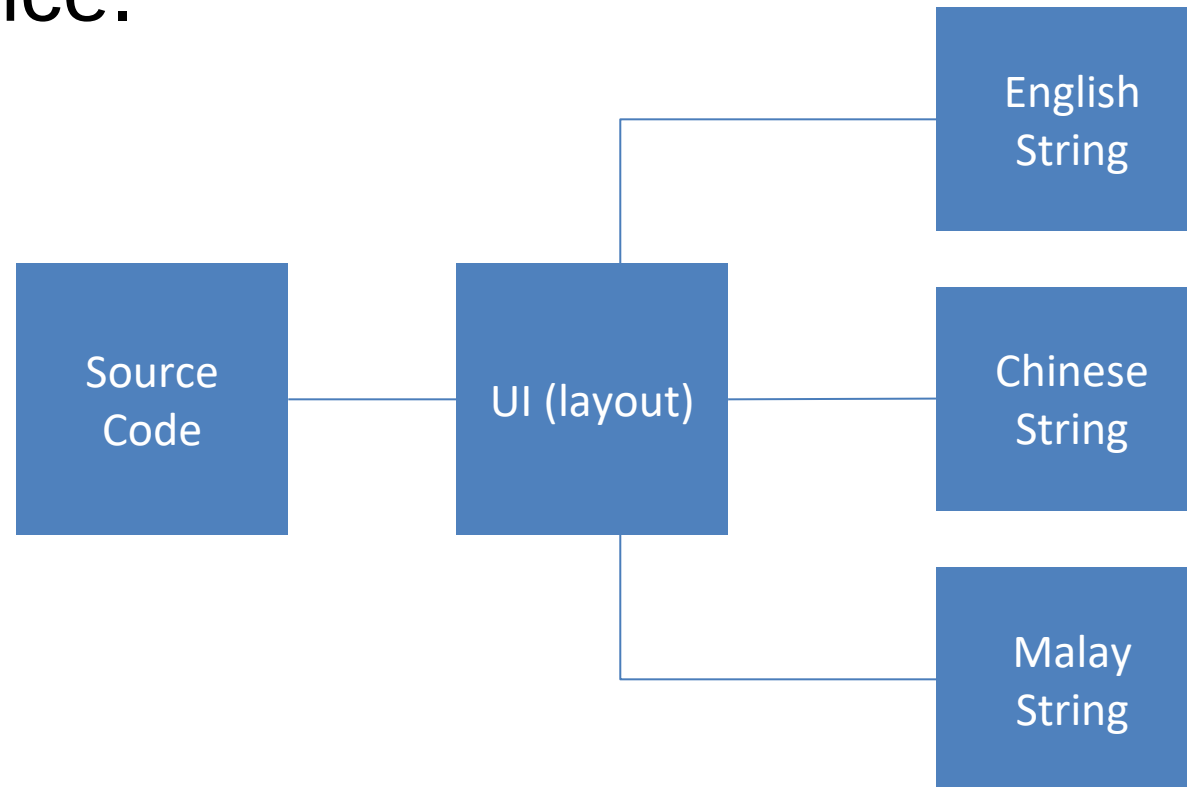
```
MyProject/  
  res/  
    values/  
      strings.xml  
    values-es/  
      strings.xml  
    values-fr/  
      strings.xml
```

Alternative Resources

```
MyProject/  
  res/  
    values/  
      strings.xml  
  values-b+es/  
    strings.xml  
  mipmap/  
    country_flag.png  
  mipmap-b+es+ES/  
    country_flag.png
```

Create Locale Directories and String Files

At runtime, the Android system uses the appropriate set of string resources based on the locale currently set for the user's device.



Create Locale Directories and String Files

English (default locale), `/values/strings.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="title">My Application</string>
    <string name="hello_world">Hello World!</string>
</resources>
```

French, `/values-fr/strings.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="title">Mon Application</string>
    <string name="hello_world">Bonjour le monde !</string>
</resources>
```

Language Code

Locale Language Code:

- English: en
- English (UK): en_GB
- English (US): en_US

- Chinese language: zh
- Chinese (China) : zh_CN

- Malay (Indonesia) : in

- Bahasa Melayu : ms

Use the String Resources

Refer to a string resource with the syntax `R.string.<string_name>`.

```
val hello: String = getString(R.string.hello)
```

Use the String-Array Resources

Refer to a string-array resource with the syntax `R.string.<string_name>`.

```
<resources>
  <string-array name="states_array">
    <item>Johor</item>
    ...
  </string-array>
</resources>
```

```
val array: Array = resources.getStringArray(R.array.planets_array)
```

Design for Everyone

Languages

Screen

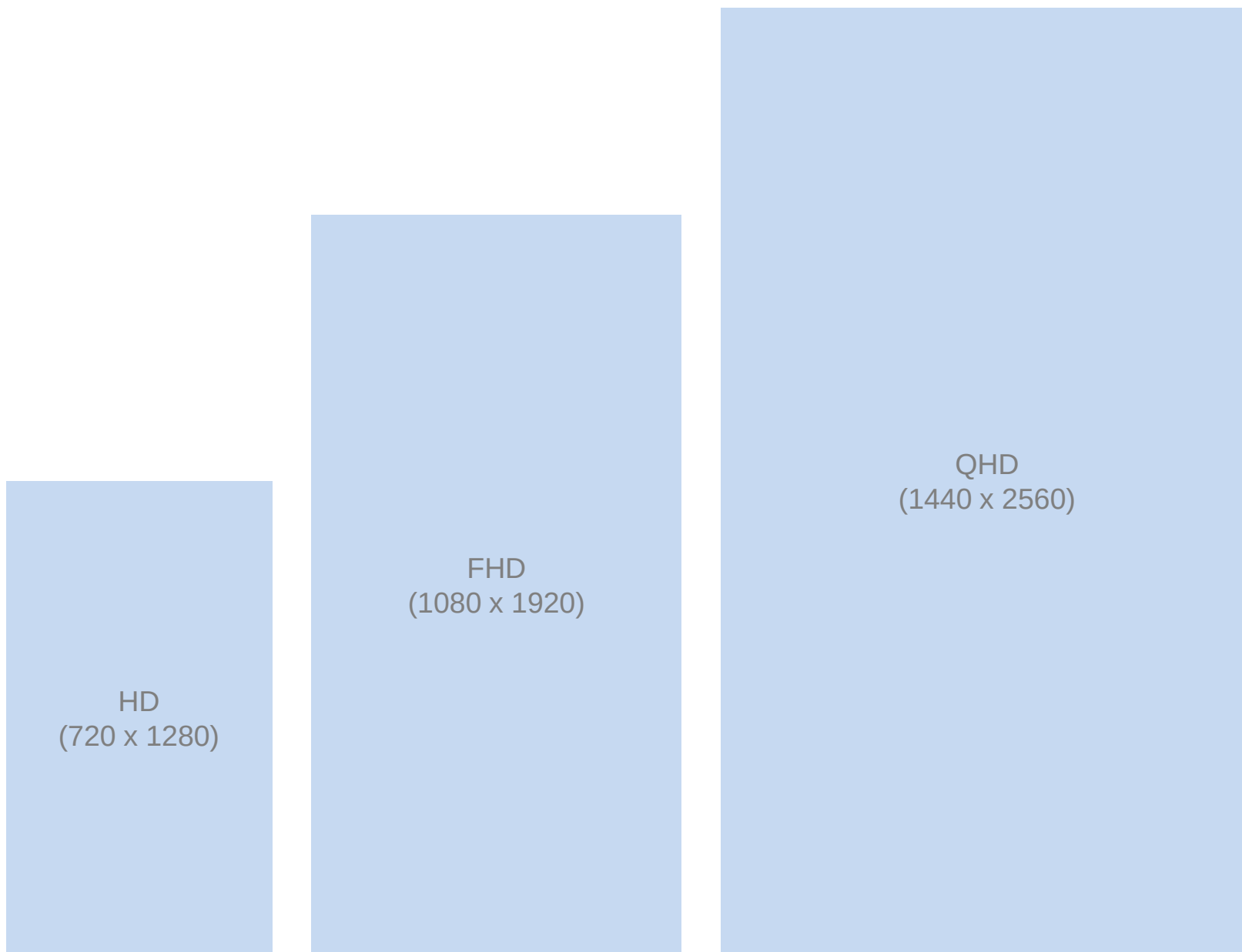
Versions of Android

Supporting Different Screens

Android categorizes device screens using two general properties:

Size
Small
Normal
Large
xLarge

Density
Low (ldpi)
Medium (mdpi)
High (hdpi)
Extra high (xhdpi)



Supporting Different Screens

Declare different layouts and bitmaps for different screens

Place these alternative resources in separate directories

Screens orientation (landscape or portrait) is considered a variation of screen size

Create Different Layouts

Create a unique layout XML file for each screen size

MyProject/
res/

layout/

Each layout should be saved into the appropriate resources directory, named with a - <screen_size> suffix.

main.xml

layout-large/
main.xml

Create Different Layouts

The file names must be exactly the same, but their contents are different in order to provide an optimized UI for the corresponding screen size.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.main)
}
```

Create Different Layouts

The system loads the layout file from the appropriate layout directory based on screen size/orientation of the device on which your app is running.

```
MyProject/  
  res/  
    layout/ # default (portrait)  
      main.xml  
    layout-land/ # landscape  
      main.xml  
    layout-large/ # large (portrait)  
      main.xml  
    layout-large-land/ # large landscape  
      main.xml
```

Jetpack Compose

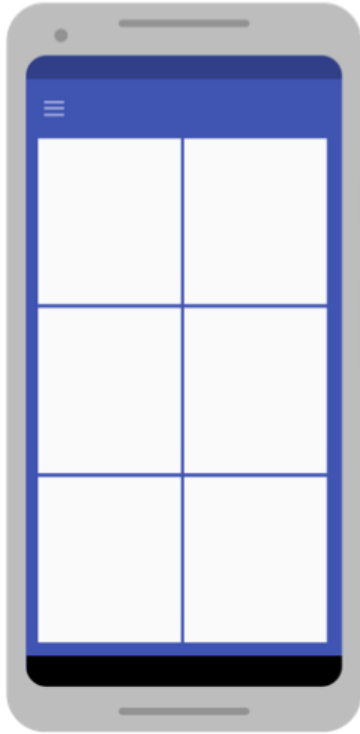
```
@Composable
fun MultiScreenLayout(windowSizeClass: WindowSizeClass) {
    when (windowSizeClass.widthSizeClass) {
        WindowWidthSizeClass.Compact -> PhoneLayout()
        WindowWidthSizeClass.Medium -> TabletLayout()
        WindowWidthSizeClass.Expanded -> DesktopLayout()
    }
}
```

```
@Composable
fun PhoneLayout() {
    Column(modifier = Modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally) {
        Text("This is a phone layout", fontSize = 18.sp)
    }
}
```

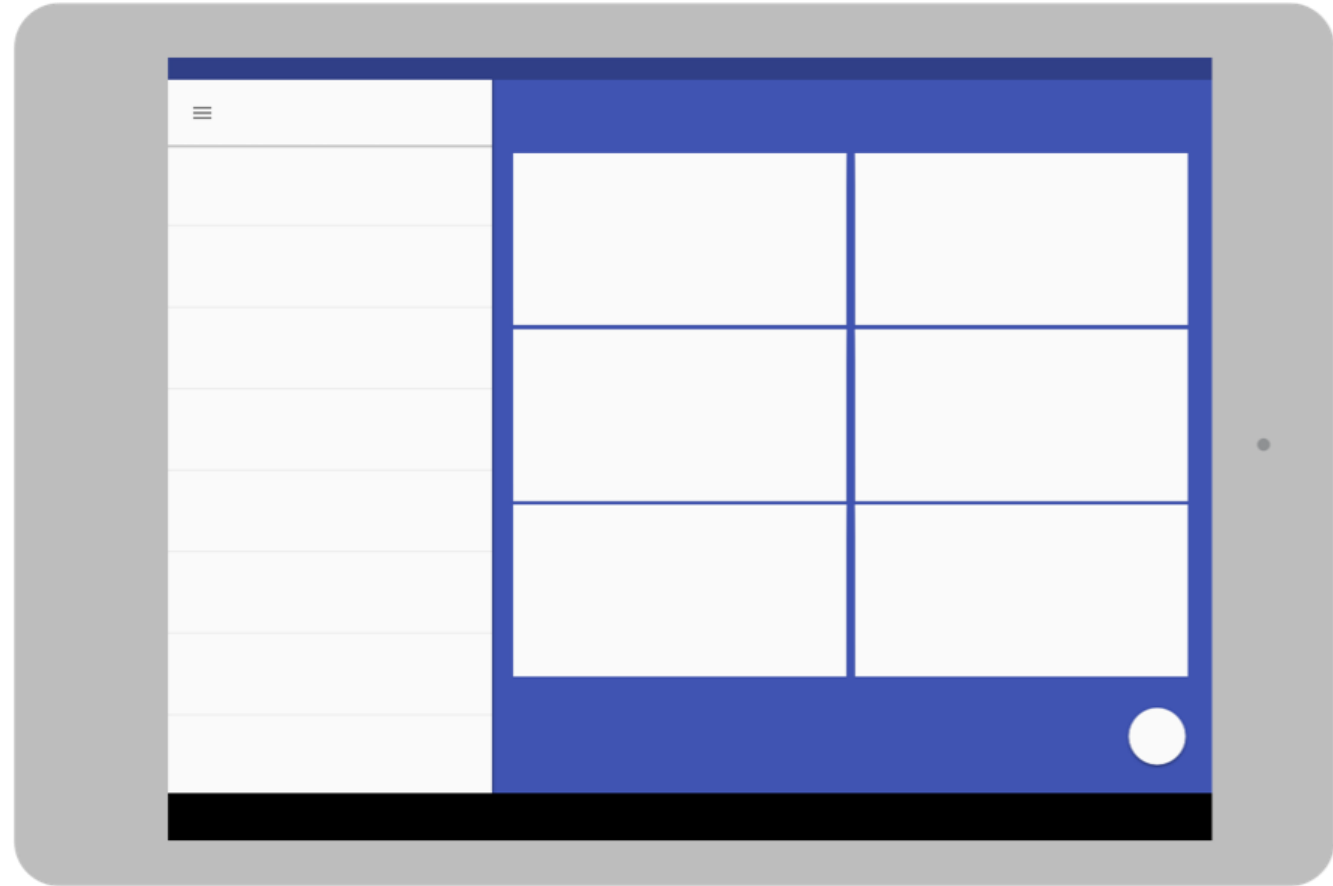
```
@Composable
fun TabletLayout() {
    Row(modifier = Modifier.fillMaxSize(), horizontalArrangement = Arrangement.Center) {
        Text("This is a tablet layout", fontSize = 20.sp)
    }
}
```

```
@Composable
fun DesktopLayout() {
    Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Text("This is a desktop layout", fontSize = 22.sp)
    }
}
```

```
dependencies {
    implementation "androidx.compose.material3:material3-window-size-class:1.1.0"
}
```



layout/



layout-land/

Support Display Cutouts



Tear Drop



Notch



Punch Hole (Middle)

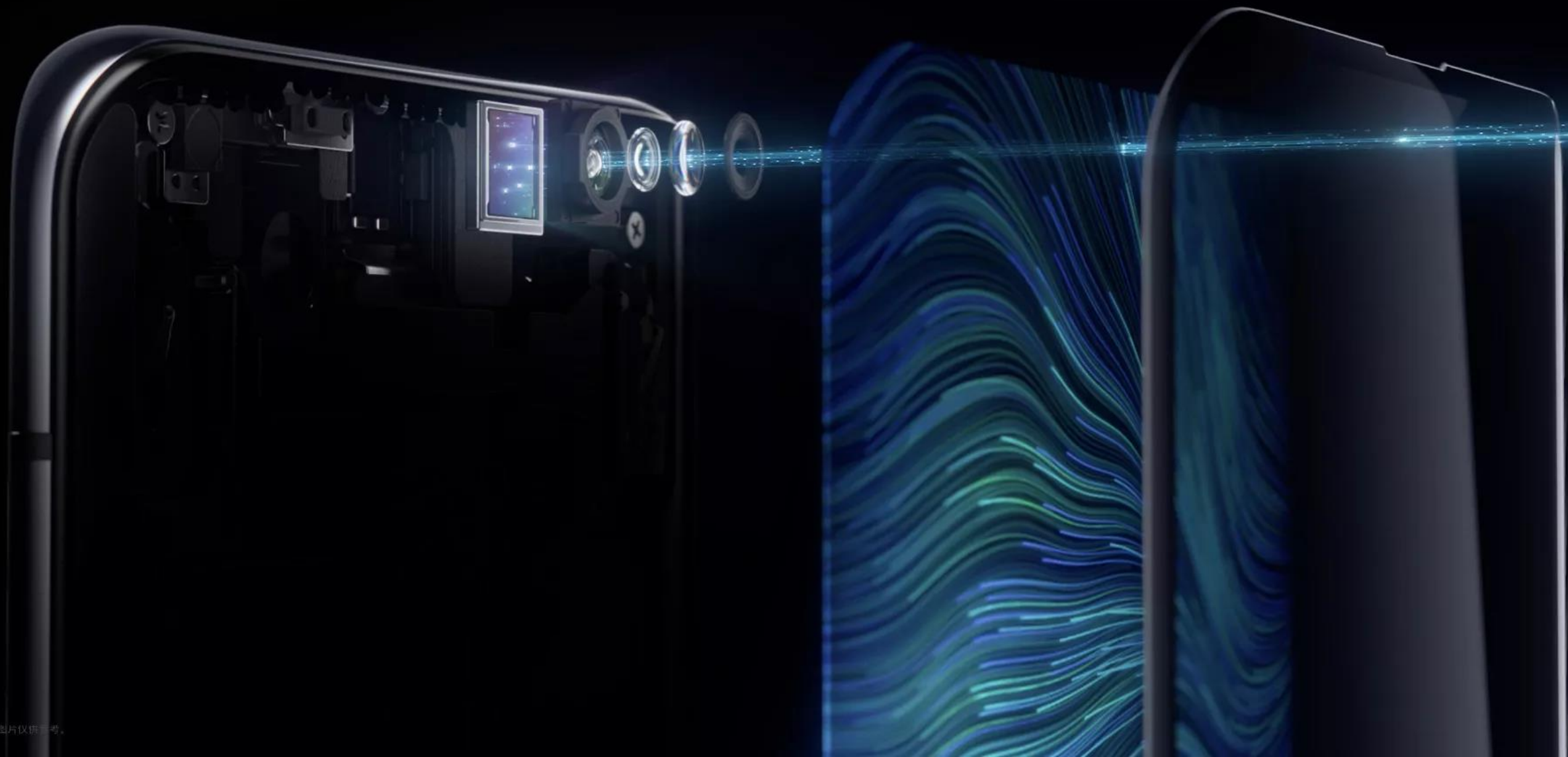


Punch Hole (Left)



Punch Hole (Right)







Support Display Cutouts

Android 9 (API level 28)

Cutout Mode

- Default
- Short Edges
- Never



```
<style name="ActivityTheme">
  <item name="android:windowLayoutInDisplayCutoutMode">
    shortEdges <!-- default, shortEdges, never -->
  </item>
</style>
```

Tablets, Large Screens, and Foldables

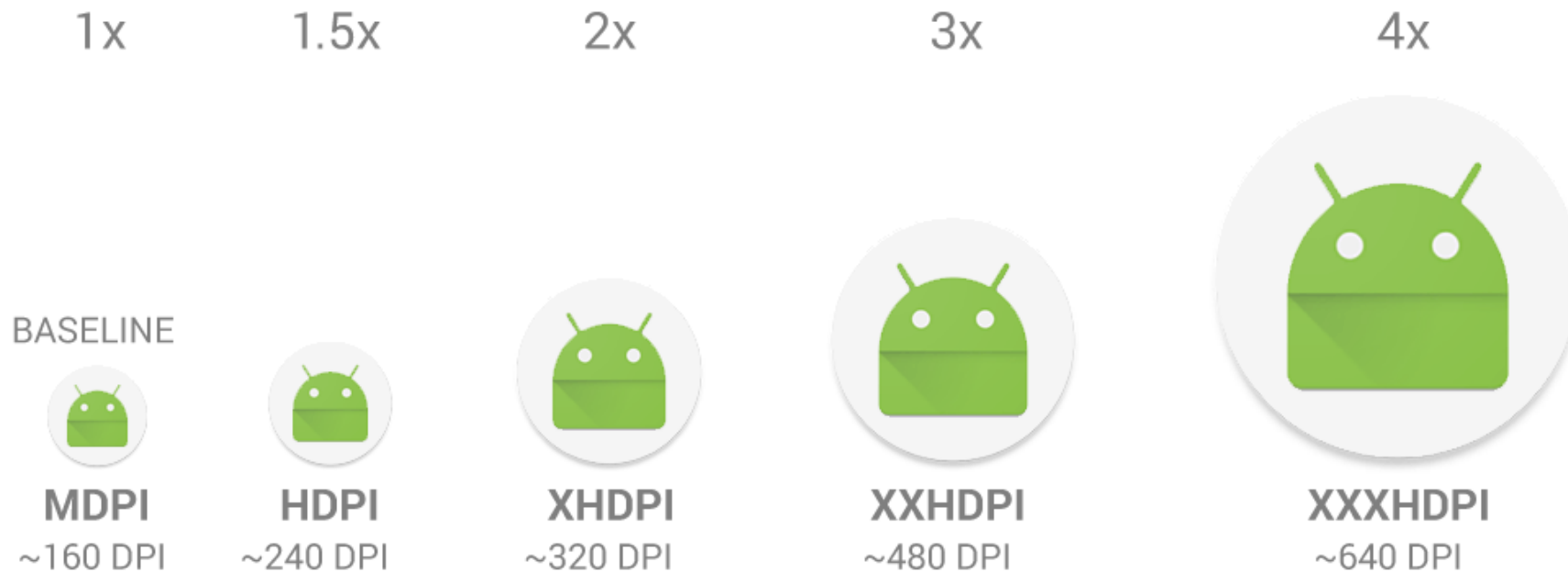


Window Manager + Folding Feature



Create Different Bitmaps

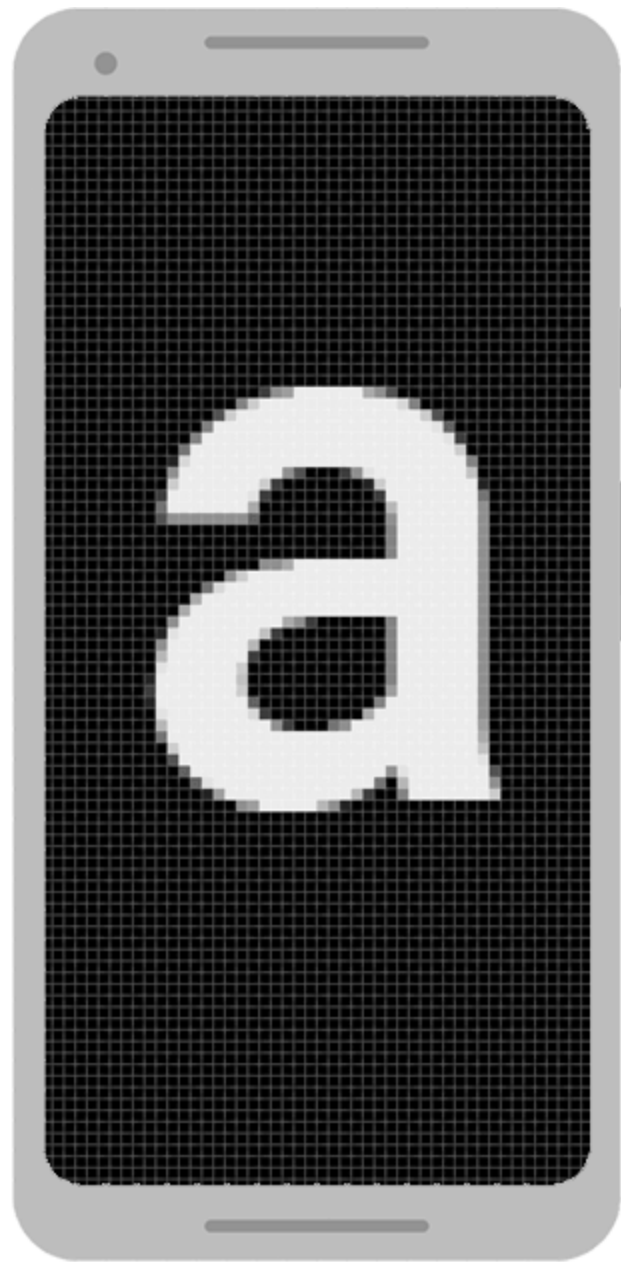
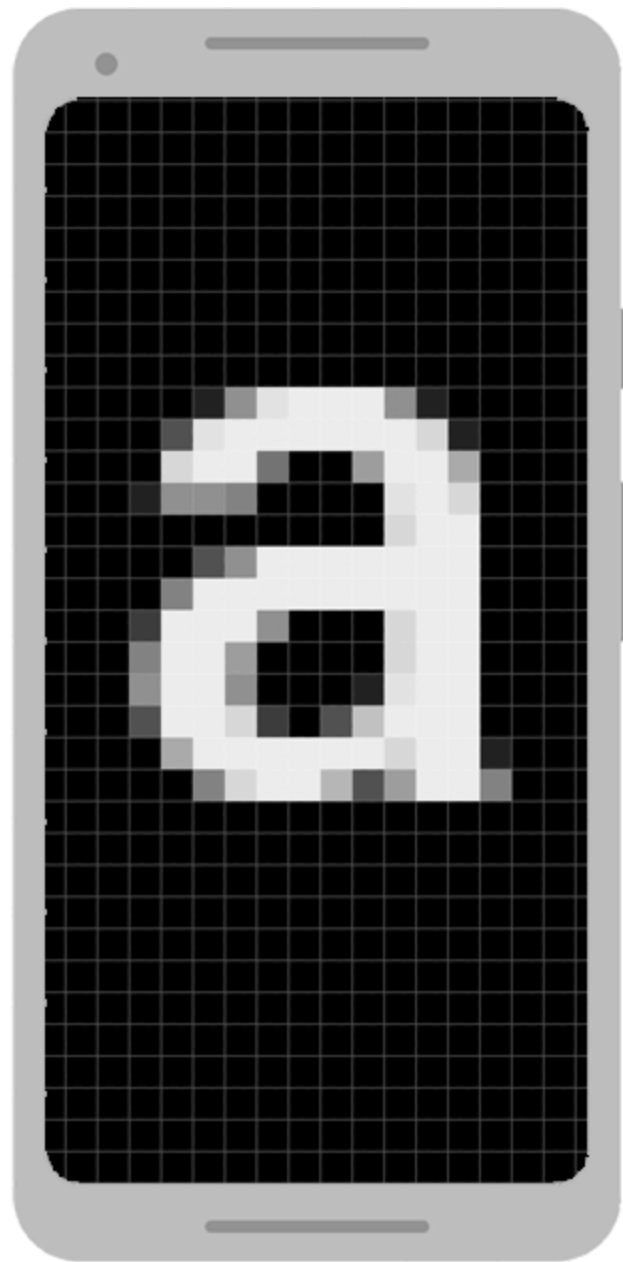
Provide alternative bitmap resources for good graphical quality and performance



Create Different Bitmaps

Place the files in the appropriate drawable resource directory:

```
MyProject/  
  res/  
    drawable-xhdpi/  
      awesomeimage.png  
    drawable-hdpi/  
      awesomeimage.png  
    drawable-mdpi/  
      awesomeimage.png  
    drawable-ldpi/  
      awesomeimage.png
```



Design for Everyone

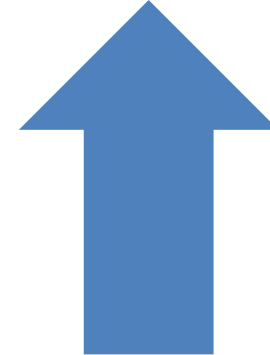
Languages

Screen

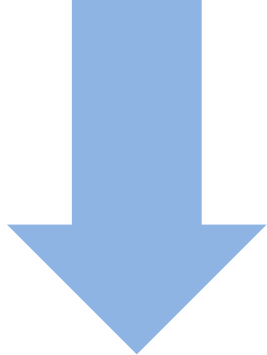
Versions of Android

Specify Minimum and Target API Levels

targetSdkVersion = the highest API level
against which you've designed and
tested your app



minSdkVersion = the lowest API level
with which your app is compatible



Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .  
  
    defaultConfig {  
        . . .  
        minSdkVersion 16  
        targetSdkVersion 30  
        . . .  
    }  
}
```



To support more Android devices

Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .
```

```
    defaultConfig {
```

```
        . . .
```

```
        minSdkVersion 16
```

```
        targetSdkVersion 30
```

```
        . . .
```

```
    }
```

To include the latest OS features

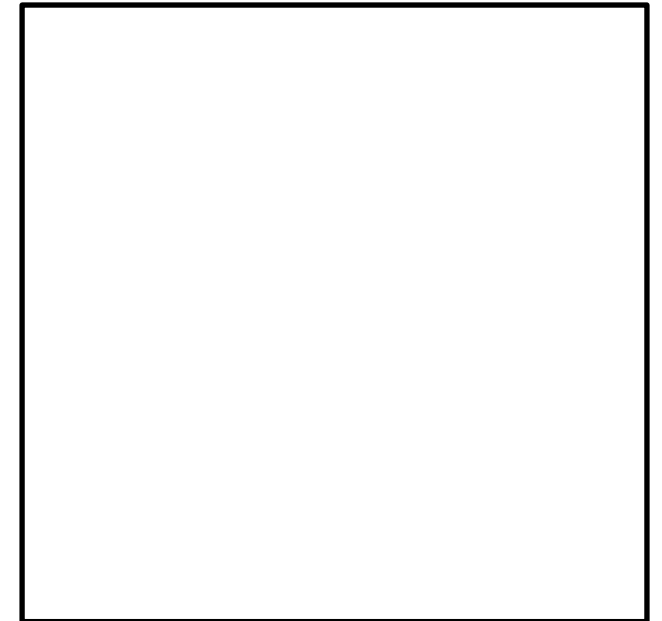
Check System Version at Runtime

In program code:

```
private void setUpActionBar() {  
    // Make sure we're running on Honeycomb  
    // or higher to use ActionBar APIs  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.HONEYCOMB) {  
        ActionBar actionBar = getActionBar();  
        actionBar.setDisplayHomeAsUpEnabled(true);  
    }  
}
```

Question?

1. What are the main challenges in developing Android app?
2. What are the resources you need to create in order to make Android app supporting different languages?
3. “In order to make an image file compatible with all screen sizes, it is necessary to create it in the best quality possible” Comment on this statement.
4. Why is it appropriate to set the minimum SDK version to the lowest version and the target SDK version to the latest version?



Make apps more accessible

Accessibility

Regardless of ability, users are able to navigate, understand, and use an app successfully

Consideration:



Navigation



Readability



Guidance and Feedback

Accessibility - Navigation



1. Support screen readers by:

- label UI elements
- group related content

```
<ImageButton  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_alignParentBottom="true"  
    android:layout_alignParentLeft="true"  
    android:layout_alignParentStart="true"  
    android:contentDescription="@string/share"  
    android:src="@drawable/ic_share" />
```

```
<ConstraintLayout  
    android:id="@+id/song_data_container"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:screenReaderFocusable="true">  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:id="@+id/song_title"  
        android:text="@string/song_title" />
```

...

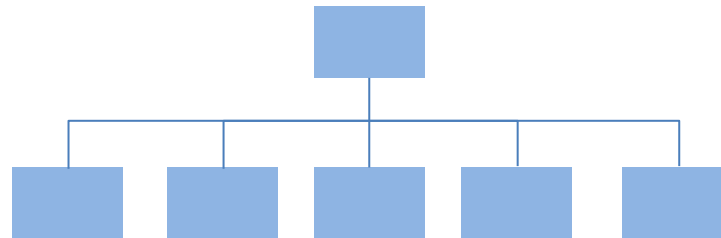
```
</ConstraintLayout>
```

Accessibility - Navigation

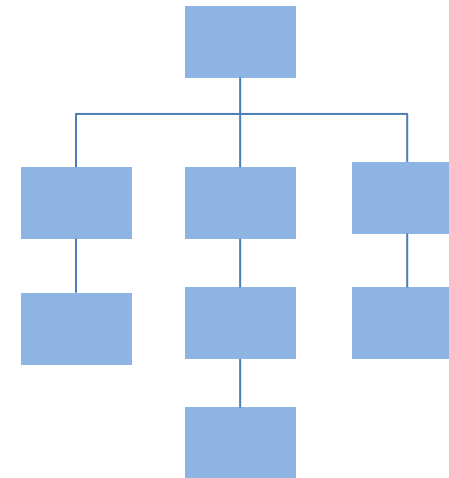


2. Create easy-to-follow navigation

- Support keyboards or gestures input
- Avoid having UI elements fade out or disappear after a certain amount of time
- Create flat navigation structure



✓ Do



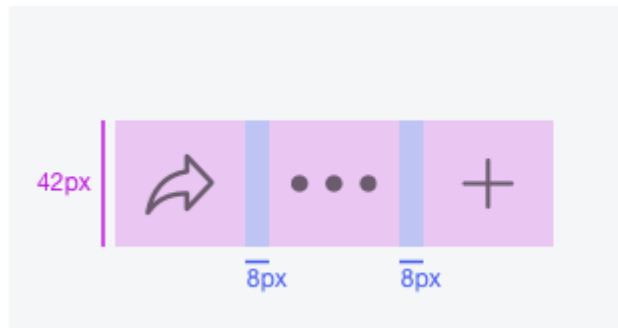
✗ Don't

Accessibility - Navigation

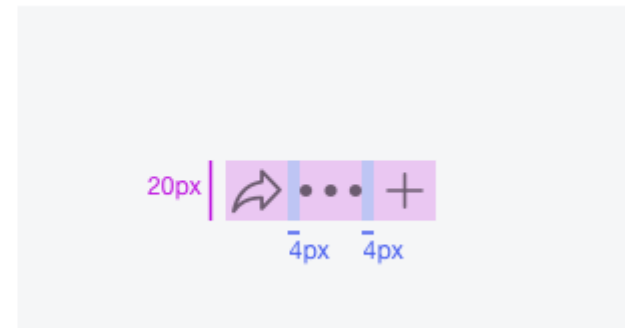


3. Make touch targets large

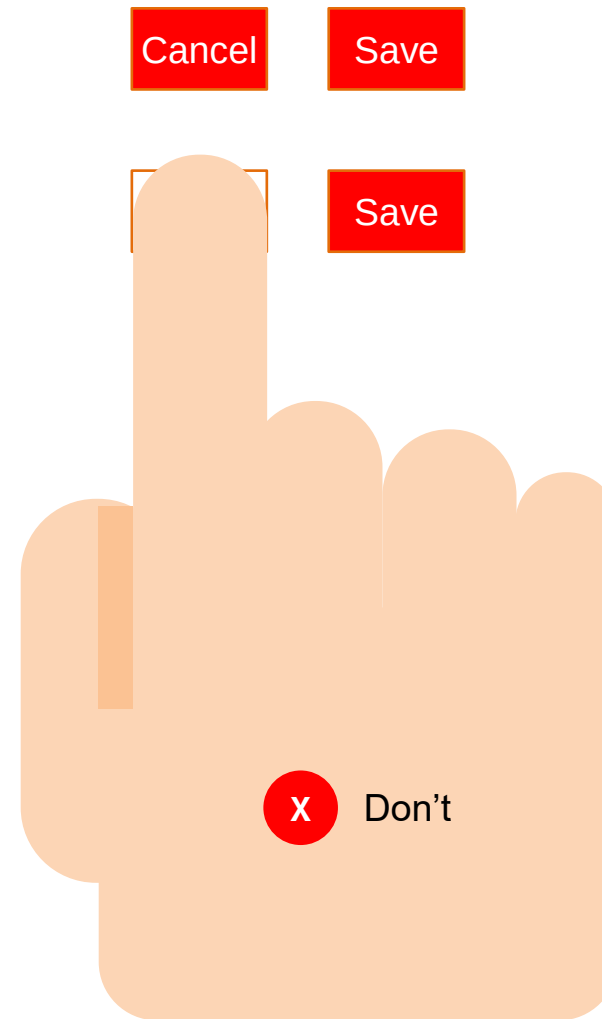
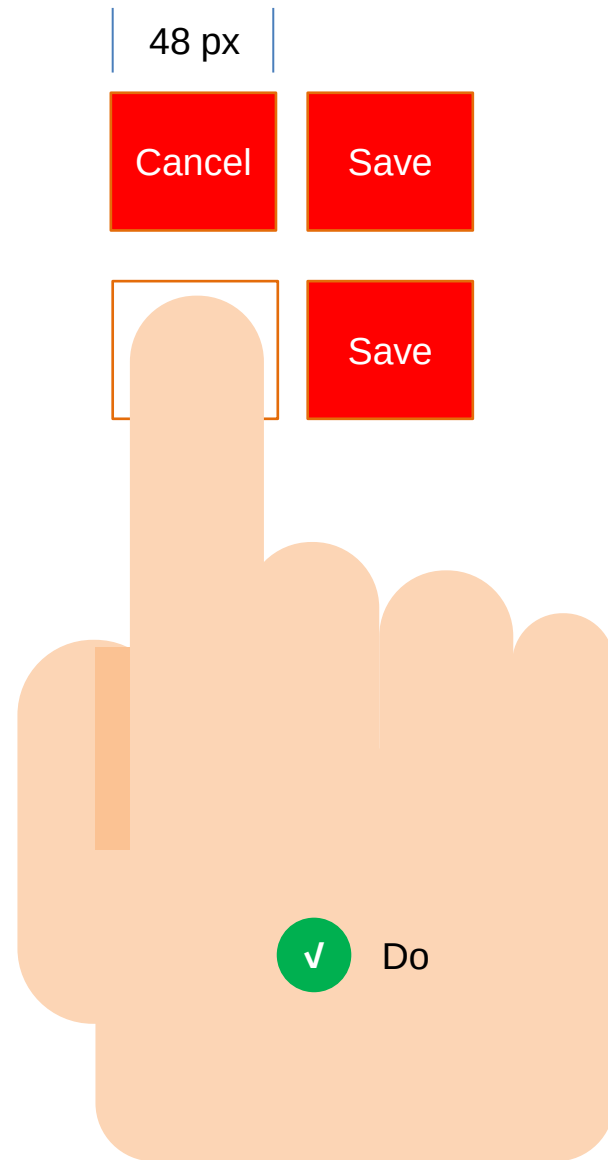
- Min size 48 x 48 pixel
- Space between elements min 8dp



Do



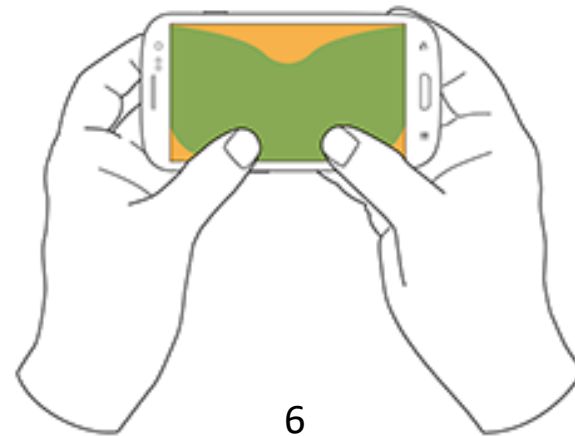
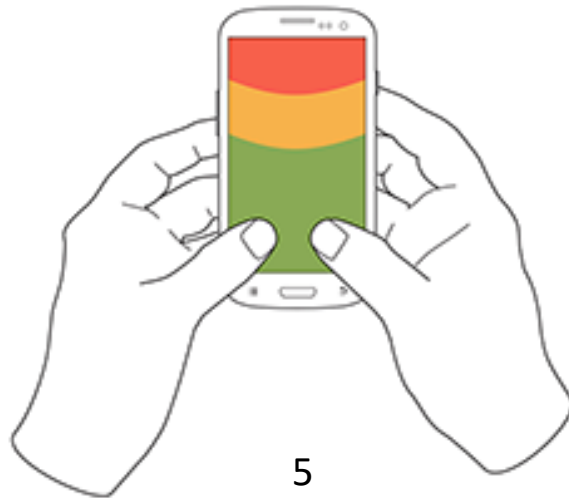
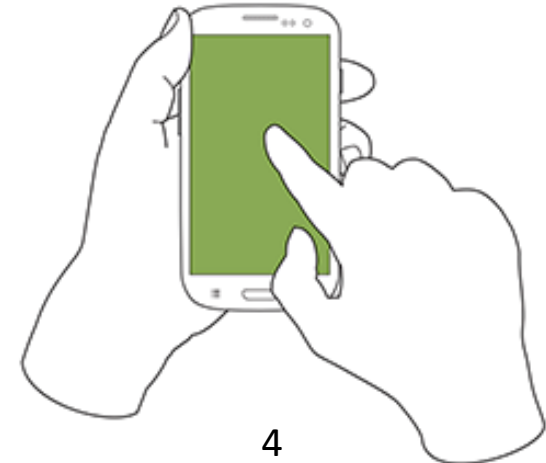
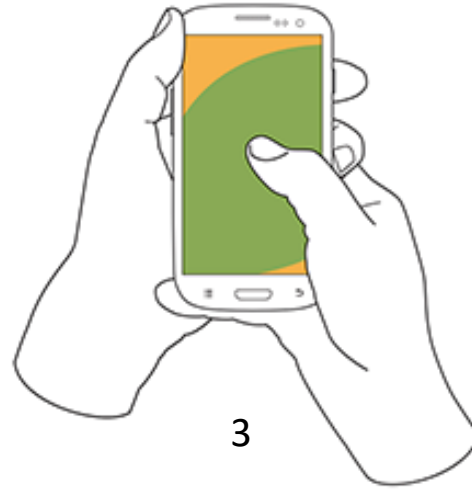
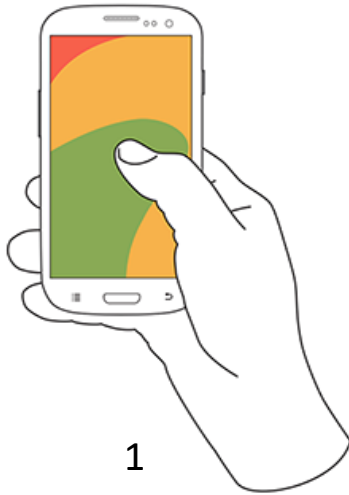
Don't



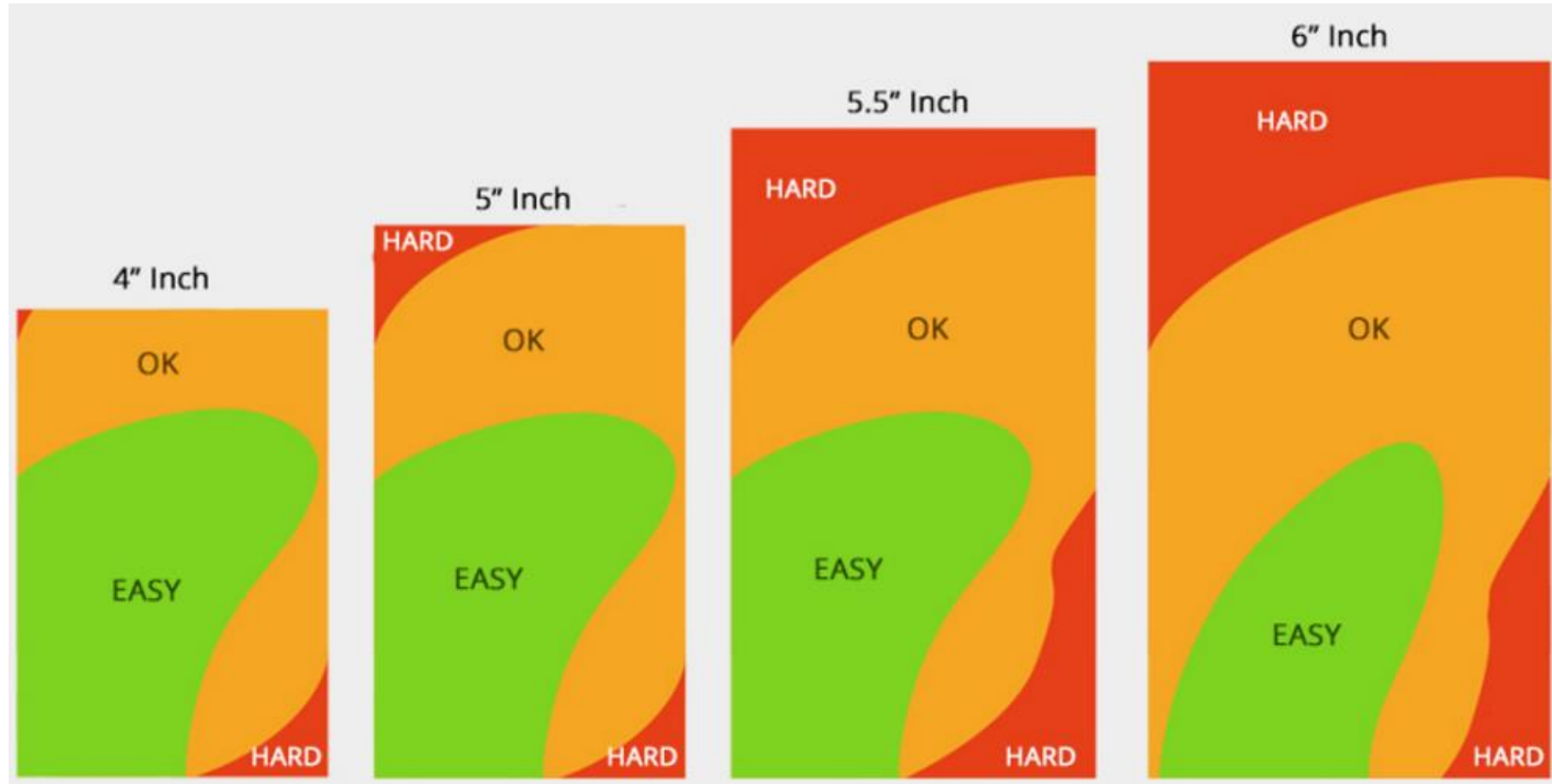
Index finger fits snugly inside.
Target edges give visual feedback.

Question?

How users hold their phones?



Touch Zone



Gesture Navigation

Android 10 (API 29) supports fully gestured-based navigation

Ensure your apps:

1. Extend content from edge to edge
2. Handle conflicting gestures

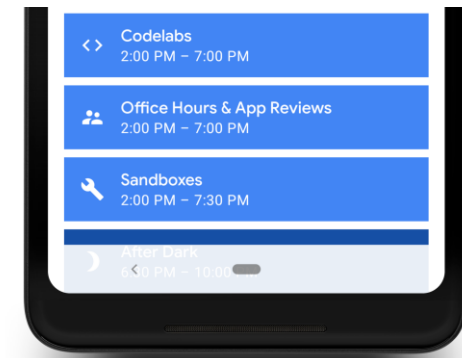
Gesture Navigation

Extend content from edge to edge

```
<!-- values-29/themes.xml: -->

<style name="AppTheme" parent="...">
    <item
name="android:navigationBarColor">@android:color/transparent</item>

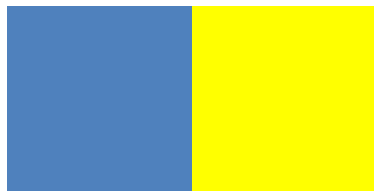
    <!-- Optional, but recommended for full edge-to-edge rendering -->
    <item name="android:statusBarColor">@android:color/transparent</item>
</style>
```



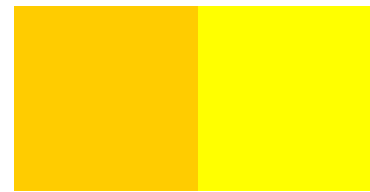
Accessibility - Readability



4. Provide adequate colour contrast

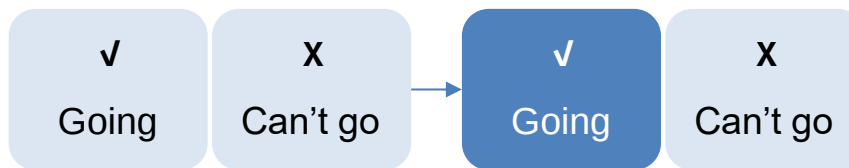


✓ Do



✗ Don't

Use more than just colour to convey information



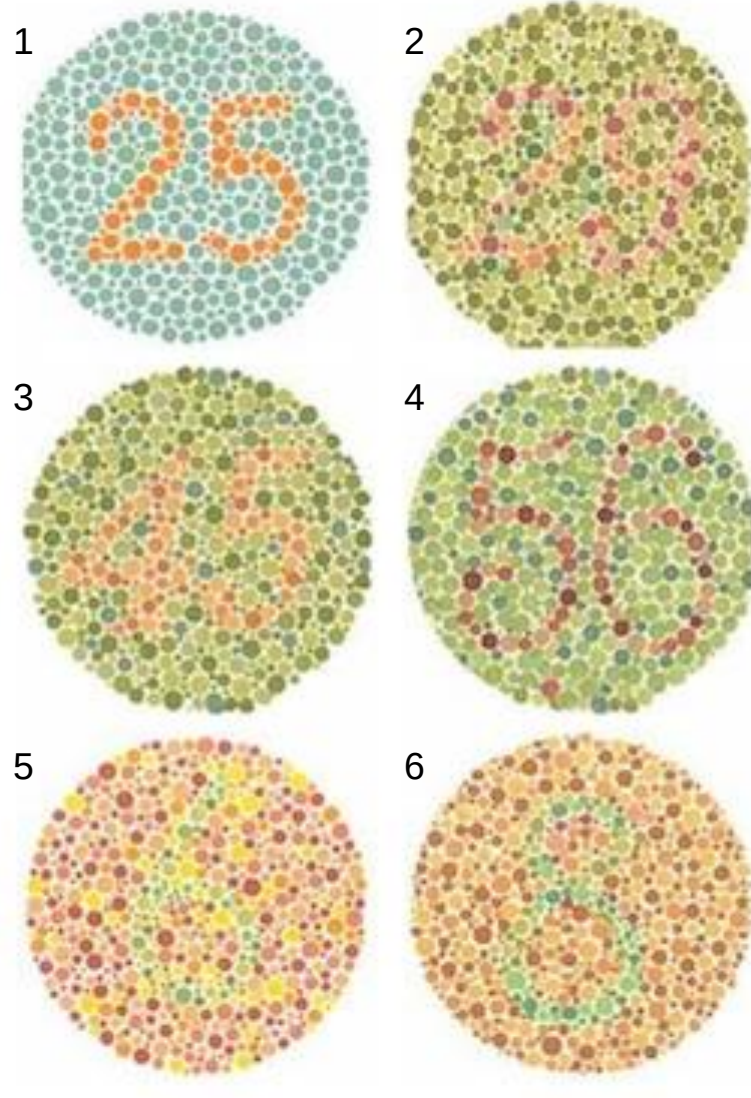
✓ Do



✗ Don't

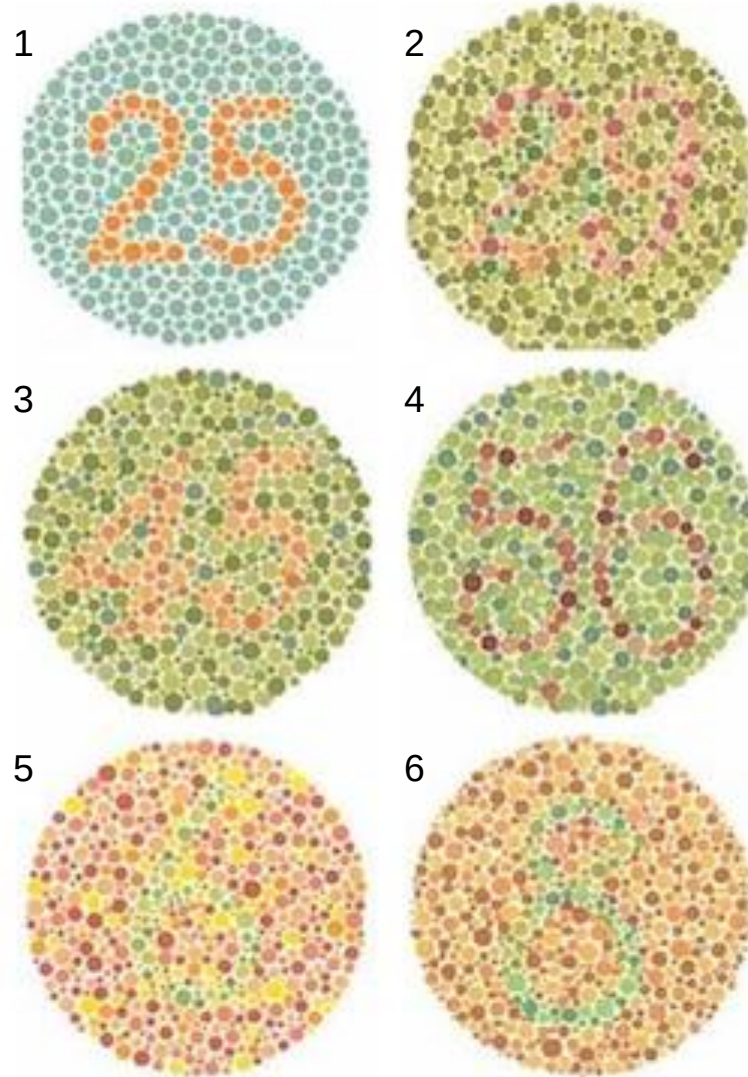
Ishihara Test For Color Blindness

What People With Regular Vision See

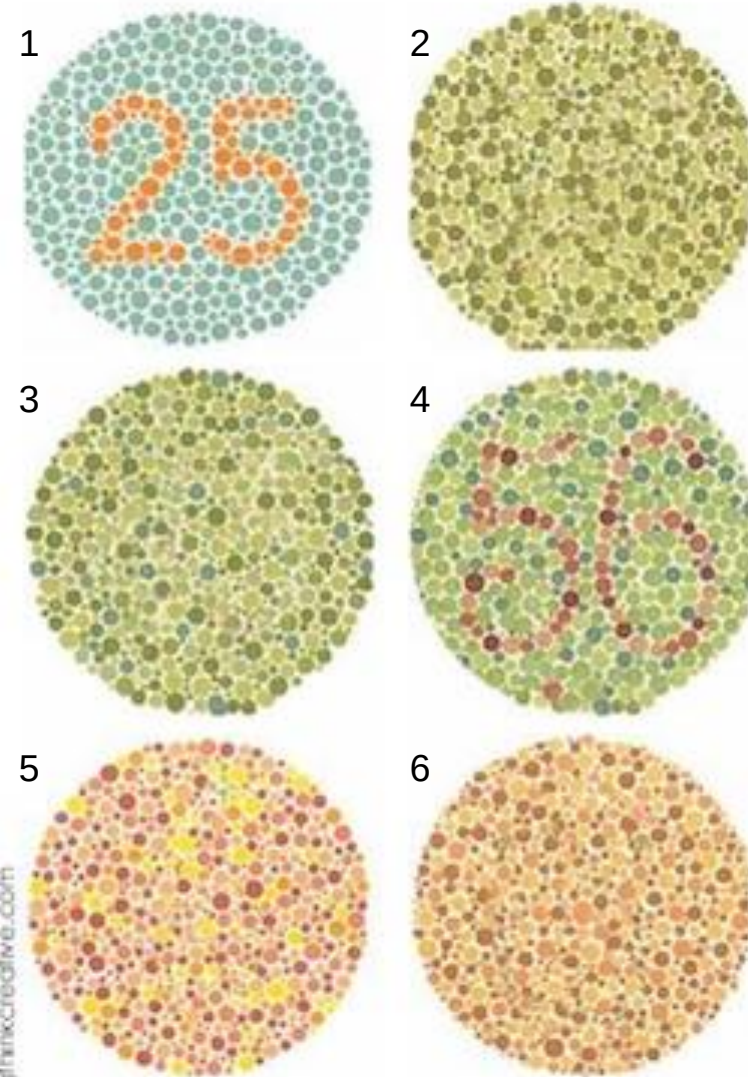


Ishihara Test For Color Blindness

What People With Regular Vision See



What People With Color Blindness See



For Normal peoples

YES

Green

NO

Red

For Deuteranopia peoples

YES

Shades of yellow

NO

For Protanopia peoples

YES

Shades of yellow

NO

For Tritanopia peoples

YES

blue

NO

Red



Accessibility - Readability



5. Make media content more accessible

- Include controls for users to pause or stop video and audio files
- Provide transcript/caption

Accessibility - Guidance and Feedback

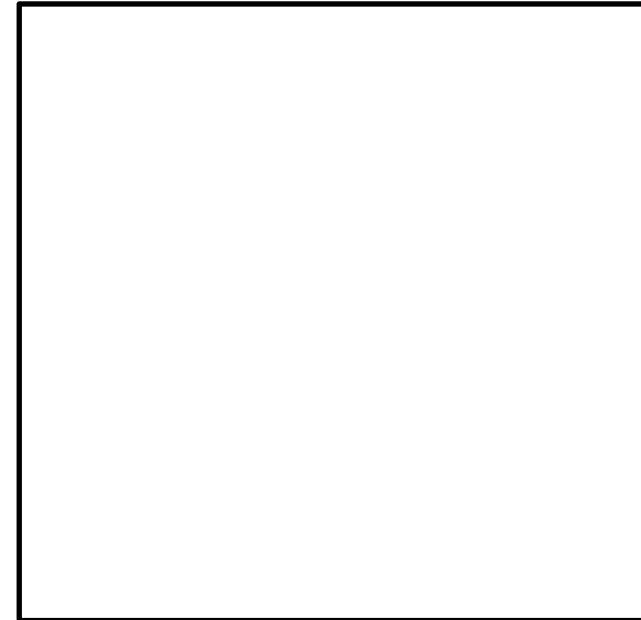


6. Make interactive controls clear and discoverable

- Interactive controls have text labels, tooltips, or placeholder text to indicate their purpose
- When naming elements, be consistent in your terminology throughout your app.

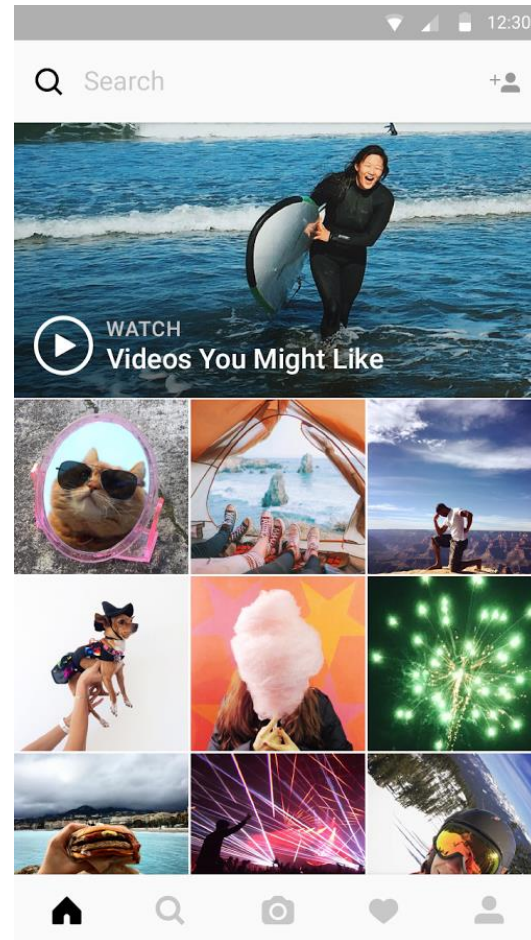
Question?

1. What is accessibility? Why it is important?
2. What is the main difference between navigation design for desktop and mobile app?
3. What is the main role of colour in UI design?



Question?

Identify spaces allocated for content and navigation



Conclusion

Create Menus (ToolBar)

Use Toasts, Snackbar and Notifications

Design UI for everyone

Make apps more accessible